Distributed Key-Value Store — System Design Document

Status: Ready to Deploy **Owner:** Somasekhar Thupakula **Last updated:** 21-07-26

---

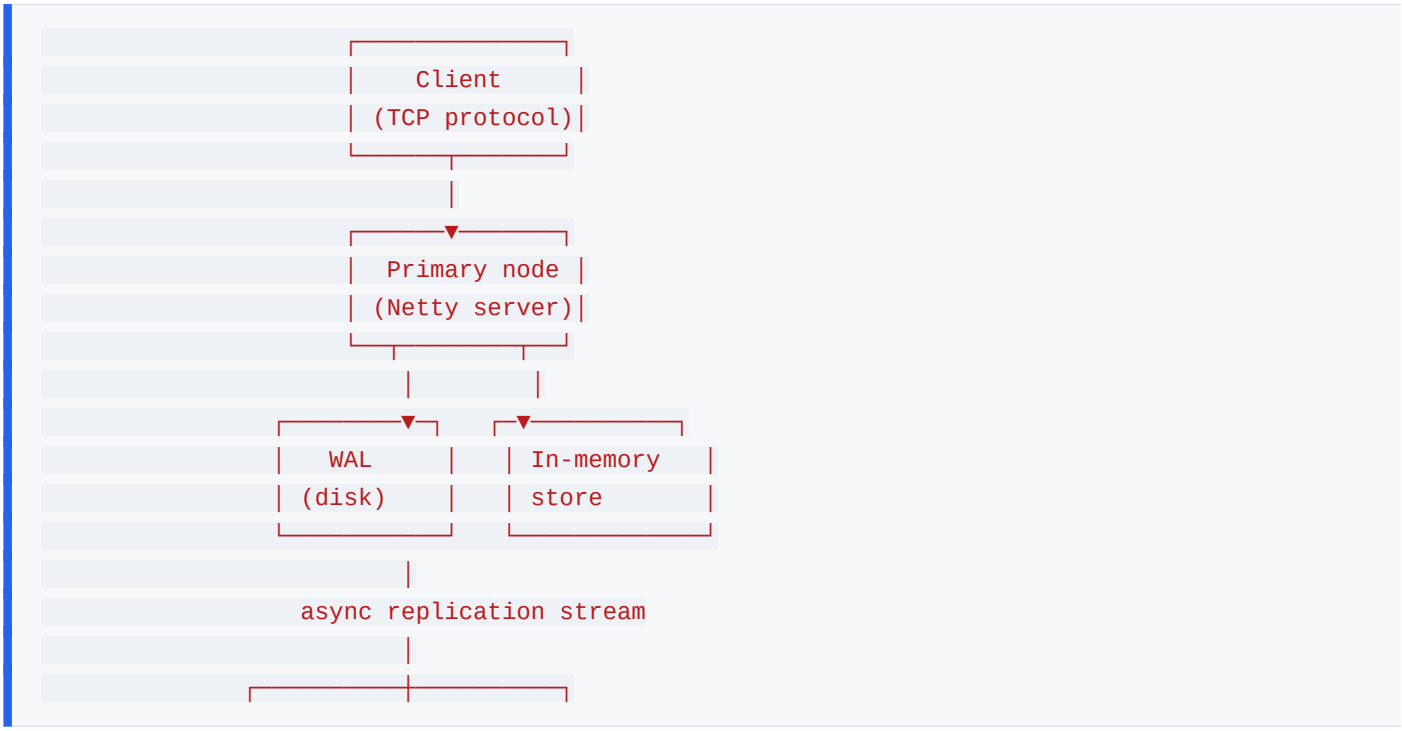
1. Executive Summary

mini-kv is a replicated, in-memory key-value store built from first principles in Java — implementing the same core mechanisms that underpin systems like Redis: a write-ahead log and snapshotting for crash durability, and asynchronous primary-replica replication for availability beyond a single node.

Key capabilities:

- In-memory `GET`/`SET`/`DEL`/`EXPIRE` over a custom TCP protocol, built on Netty
- Crash-safe durability via write-ahead logging with disk flushes before acknowledgment
- Bounded recovery time via periodic, atomic snapshotting
- Asynchronous primary-replica replication with automatic client reconnection and live lag tracking
- Documented, explicitly tested consistency model — including what data can be lost, and under what failure conditions

2. System Architecture Overview



```
        ┌──────▼──────┐           ┌──────▼──────┐
        │ Replica 1   │           │ Replica 2   │
        │ (lag tracked)           │ (lag tracked)
        └─────────────┘           └─────────────┘
```

Clients connect directly to the primary over TCP for reads and writes. Every write is logged to the WAL before being applied to the in-memory map, then streamed asynchronously to any connected replicas, each of which applies the same sequence independently and tracks how far behind it is.

3. Component Breakdown

3.1 "Frontend" — Status Dashboard

This project has no end-user-facing application frontend; its closest analog is a lightweight status dashboard (a single HTML page or simple React view) exposing live replication lag, key count, and node uptime per node.

Key pages:

- Node status overview (role, uptime, key count)
- Replication lag per replica

3.2 Backend — Java 21 / Netty

The entire system *is* the backend — there's no separate application-layer service consuming it in isolation. Structured by concern: `server/` (networking), `store/` (core map + expiry), `wal/`, `snapshot/`, `replication/`.

3.3 Database — None (self-contained persistence)

mini-kv does not depend on an external database; it *is* the storage engine. Durability is handled entirely by its own WAL and snapshot files on local disk.

3.4 Reverse Proxy — nginx

Not required for client traffic (clients speak the TCP protocol directly), but nginx (or a simple reverse proxy) fronts the optional HTTP status dashboard if one is exposed publicly.

4. Data Model

Core Entities

mini-kv has no relational schema — its "entities" are protocol-level concepts, not database tables:

```
StoredValue
  value: string, expiresAt: nullable timestamp
```

```
WalEntry
  sequencePosition, commandType [SET|DEL|EXPIRE], key, value, timestamp

Snapshot
  a full point-in-time serialization of the in-memory store, written atomically
```

Key Relationships

- Every `WalEntry` corresponds to exactly one mutation applied to the in-memory store
- A `Snapshot` represents the cumulative effect of all `WalEntry` records up to the point it was taken, allowing everything before it to be truncated

5. API Design

Not a REST API — a line-based TCP protocol:

```
SET key value       → OK
GET key             → value | (nil)
DEL key             → OK
EXPIRE key seconds  → OK
STATUS              → role, uptime, key count, (if replica) lag
```

(Documented here as the equivalent of an "API design" section for a protocol-based system rather than an HTTP one.)

6. Authentication & Authorization

6.1 Local Authentication *(Bootstrap / Temporary)*

Not applicable — mini-kv currently has no authentication layer; it assumes a trusted network (e.g. internal Docker network or VPC), the same assumption Redis makes by default without `requirepass` configured.

6.2 OAuth2 Authorization Code Flow *(Production Auth)*

Not applicable to this project — there is no end-user identity concept, only trusted service-to-service connections (a consuming application, or replication between nodes).

6.4 Role Permissions Summary

Actor	Permitted actions
Any connected client	Full `GET`/`SET`/`DEL`/`EXPIRE` on any key
Replica node	Read-only application of the replication stream from its configured primary

(Flag explicitly in Known Limitations: lack of auth/ACLs is a real gap if this is ever exposed beyond a trusted

network — see Section 11.)

7. Write & Replication Lifecycle

```
Client write → WAL append (disk flush) → apply to in-memory store → ack to client
                                                  │
                                                  └──→ async stream to replicas → each replica applies + tracks lag
```

A write is only ever acknowledged to the client after it is durably logged — this ordering is the core crash-safety guarantee. Replication is intentionally asynchronous and happens after acknowledgment, which is the source of the project's one explicitly accepted risk: a write acknowledged to a client can be lost if the primary crashes before that entry reaches any replica. This tradeoff, and its tested behavior under a killed-primary chaos test, is documented in `ARCHITECTURE.md`.

8. Infrastructure & Deployment

8.1 Environment

Local development and multi-node testing via Docker Compose (1 primary + 2 replicas). Production-style deployment target: small VMs (e.g. Oracle Cloud free-tier ARM instances), one per node, chosen to demonstrate genuine multi-host replication rather than same-host containers only.

8.2 Container Configuration

Single Dockerfile, parameterized by `NodeConfig` (role: primary/replica, peer address) via environment variables, so the same image serves both roles.

8.3 Networking

Nodes communicate over a dedicated replication port, separate from the client-facing port. In a multi-VM deployment, this traffic should be restricted to a private network/VPC, not exposed publicly.

8.4 Secrets Management

Minimal at present — no credentials are required between trusted nodes currently. If authentication is added (see Known Limitations), shared secrets would be managed the same way as the other two projects: environment variables, never committed.

8.5 Database Backup

(Not a traditional database, but the equivalent concern applies to snapshot files — document the actual backup/retention approach for snapshot files once the deployment target is finalized, e.g. periodic copy to object storage.)

8.6 Deployment Process

GitHub Actions: build, unit tests, concurrency stress tests, and crash-recovery tests gate every merge; chaos tests (kill primary mid-stream) are run as part of the Phase 4 exit criteria before any deploy, per the project's TASKS.md.

9. Security Design

The current design assumes a trusted network boundary and has no authentication layer — this is an explicit, documented limitation rather than an oversight (see Section 11). The primary "security" property this project focuses on instead is data-integrity under failure: crash-safety (no acknowledged write lost to a process crash) and a precisely documented, tested consistency model for replication (exactly what can be lost, and when).

10. Non-Functional Requirements

- **Durability:** no acknowledged write is lost on process restart, verified via `kill -9` recovery tests
- **Bounded recovery time:** restart time driven by current dataset size, not historical write volume, verified via a snapshot-vs-no-snapshot recovery benchmark
- **Replication correctness:** replicas apply writes in the same order as the primary, verified via the chaos test suite
- **Observability:** replication lag must be visible and accurate at all times a replica is connected

11. Known Limitations & Future Considerations

- No authentication/ACL layer — acceptable only within a trusted network; a real deployment beyond a private VPC would need this addressed first
- No automatic failover — promotion of a replica to primary is a manual/reviewed action by design, not an oversight
- No sharding — a single primary's write throughput is the ceiling for this design
- Candidate for integration as the seat-hold backend for the ticket booking project, gated behind that project's full concurrency and chaos test suite before any production cutover